Lab 17 — Answers with commentary

Every answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — Chart, release, revision

Answer. The **chart** is the package: templates, default values and metadata, in a directory or a `.tgz`. A **release** is one installation of that chart under a name, in a namespace (`stack` in `lab17`): the same chart can be installed several times under different names. A **revision** is a numbered version of the release: every `install`, `upgrade` or `rollback` creates one. Helm stores each revision (chart + values + rendered manifest) in a Secret `sh.helm.release.v1.stack.v<n>` of the namespace. Since everything lives in the API server, a client alone, with a kubeconfig, retrieves the history and can reapply a revision: no Tiller, no database.

Why. Helm 3 replaced the Tiller server with this in-cluster storage, under the user's RBAC rights.

Nuance. By default Helm keeps 10 revisions (`--history-max`). Deleting those Secrets by hand destroys the history; `helm uninstall` deletes them too (except with `--keep-history`).

Example.

```
kubectl get secrets -n lab17 -l owner=helm      # sh.helm.release.v1.stack.v1, .v2, …
helm history stack -n lab17
```

Question 2 — `--set` and forgotten values

Answer. `helm get values` prints the **user-supplied values** of the last revision: only `api.message: v2`. After the third command the API runs with **2 replicas** (the `values.yaml` value), not 3: every `helm upgrade` starts again from the chart's defaults plus what you pass **on that command**. To accumulate: `helm upgrade stack ./stack --reuse-values --set api.message=v2`, or pass both `--set` again, or — better — put everything in a `values-<env>.yaml` passed every time with `-f`.

Why. `--set`s are not persistent by themselves; they are only "remembered" in the revision, and a new upgrade inherits them only with `--reuse-values`.

Nuance. `--reuse-values` is treacherous in turn: it ignores the new defaults of an updated chart. Company practice: one versioned values file per environment, and never a hand-typed `--set` in production.

Example.

```
helm upgrade stack ./stack --set api.replicas=3 ; helm upgrade stack ./stack --set
api.message=v2
kubectl get deployment api -o jsonpath='{.spec.replicas}'    # 2
helm get values stack                                        # api: message: v2
```

Question 3 — Helm cannot find the cluster

Answer. `helm` looks for a kubeconfig in the standard place (`~/.kube/config` or `$KUBECONFIG`); there is none, so it tries the default address `localhost:8080`, where nothing listens. Fix: `export`

`KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (or copy that file to `~/.kube/config`). The k3s `kubectl` does not have the problem because it is **embedded in the k3s binary**, which knows its own kubeconfig.

Why. Helm is a generic client, like Lens, k9s or a separately installed `kubectl`; they all read `KUBECONFIG`.

Nuance. The file is readable thanks to `--write-kubeconfig-mode 644` (lab 11); on a company cluster the kubeconfig is handed to you by the platform team and contains credentials: never commit it.

Example.

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml ; helm list -A    # traefik, traefik-crd
```

Question 4 — The `checksum/config` annotation

Answer. On `helm upgrade --set api.message=other`, the `configmap.yaml` template is rendered with the new message; its `sha256sum` changes; the Pod template annotation therefore changes too; since the Pod template changed, the Deployment starts a **rolling update** and the new Pods read the new ConfigMap. Without the annotation, the ConfigMap would be updated but the Deployment, unchanged, would keep its Pods with the old environment (lab 14): a `kubectl rollout restart` would be needed. Unwanted case: the ConfigMap contains something that varies for no functional reason (a timestamp, a comment, key ordering): every upgrade redeploys for nothing.

Why. The Deployment only watches its own `spec.template`; the annotation "imports" the ConfigMap's content into it as a fingerprint.

Nuance. The same technique applies to Secrets (`secret.yaml`). An alternative is a dedicated controller (Reloader) that watches ConfigMaps and restarts the Deployments mounting them.

Example.

```
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo
helm upgrade stack ./stack --reuse-values --set api.message=other
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo   # different
```

Question 5 — Rollback and PVC

Answer. After `helm rollback stack 1`, the history counts **four** revisions: 1, 2, 3 `superseded` and 4 `deployed`, described as `Rollback to 1`. Helm erases nothing: going back is a **new revision** whose content is revision 1's. After `helm uninstall`, `kubectl get pvc` still shows `data-db-0` `Bound`: Helm deletes the objects it created, and the PVC was created by the **StatefulSet** through `volumeClaimTemplates`, not by Helm. That is deliberate: uninstalling a release must not destroy a database. The data goes away with `kubectl delete pvc data-db-0` or by deleting the namespace.

Why. Same logic as `podman compose down` without `-v` and as `kubectl delete statefulset` (lab 15).

Nuance. A chart can declare hooks or the `helm.sh/resource-policy: keep` annotation to protect other objects. And `helm rollback` does not bring back a database's content: it only manages

manifests.

Example.

```
helm history stack | tail -1     # 4  …  deployed  …  Rollback to 1
helm uninstall stack ; kubectl get pvc    # data-db-0   Bound
```

Question 6 — Helm or Kustomize

Answer. (a) PostgreSQL and Prometheus: **Helm**, with the official charts (Bitnami, prometheus-community) and a `values.yaml` per environment: this software is parameterised by hundreds of options, maintained by others, and versioned like packages. (b) Four in-house microservices in three environments through Argo CD: **Kustomize**: one base per service, one overlay per environment, all readable as plain YAML in Git, no templates to debug, and Argo CD applies it natively. They coexist because every team does both: install third-party products and deploy its own; Argo CD can render a Helm chart anyway, and `helm template | kustomize` lets you patch a public chart without forking it.

Why. Helm optimises **distribution** (package, version, repository); Kustomize optimises the **readability** of variants of YAML you own.

Nuance. A very simple in-house chart (like `stack/`) and a Kustomize overlay are equivalent; the Helm/Kustomize quarrel is mostly a team matter. The real criterion: who maintains the YAML, and how many environments.

Example.

```
helm template stack ./stack -f stack/values-prod.yaml | grep -c '^kind:'    # 11 rendered
objects
kubectl kustomize kustomize/overlays/prod | grep -c '^kind:'                 # 3
```

Question 7 — Label missing on Pods

Answer. Kustomize's `labels` transformer, with `includeSelectors: false` and nothing else, only adds the label to the **objects' metadata** (Deployment, Service), not to the Pods' `spec.template`. You need `includeTemplates: true` for it to reach Pod templates too. `includeSelectors: true` would add the label **to the selectors** (`spec.selector.matchLabels` of the Deployment, `spec.selector` of the Service): but a Deployment's selector is **immutable**; on an existing Deployment the `apply` would fail (`field is immutable`), and you would have to delete and recreate it.

Why. Kustomize distinguishes three targets for a label: the object, its Pod template, its selectors; the old `commonLabels` touched all three, hence its deprecation.

Nuance. A *selector* label must stay stable for the Deployment's whole life; an *informational* label (`env`, `team`, `version`) goes on the object and the template, never in the selector.

Example.

```
labels:
  - pairs: { env: prod }
    includeSelectors: false
    includeTemplates: true
```

```
kubectl get pods -n lab17-prod -L env     # ENV prod
```

Question 8 — Two digests, one image

Answer. The local digest (`sha256:e1b2…`) is the fingerprint of the **manifest as Podman holds it locally**; on push, Podman **recompresses** the layers and produces a different manifest for the registry, whose fingerprint is `sha256:b820…`: that is what `--digestfile` writes and what the node sees in `imageID` after the pull. Both name the same content, but only the **registry's** digest is one a cluster can pull: that is the one to pin (`image: localhost:5000/api@sha256:b820…`). A tag is not enough because it is mutable: a new push on `api:1.1` silently changes what `1.1` means.

Why. A digest is the hash of a precise manifest (list of compressed layers + config); changing the compression changes the manifest, not the image once decompressed (same `IMAGE ID`, lab 10).

Nuance. `podman pull` from the registry followed by `podman image inspect` gives `b820…` this time: the local digest becomes the registry's. `skopeo inspect docker://…` queries the registry without pulling anything.

Example.

```
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 ; cat digest.txt
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'    # …@sha256:b820…
```

Question 9 — HTTP for a non-local registry

Answer. containerd (like Podman) tolerates **plain HTTP only for `localhost` / `127.0.0.1`**; for any other host it requires TLS, hence `server gave HTTP response to HTTPS client`. On k3s, you declare the registry in `/etc/rancher/k3s/registries.yaml`:

```
mirrors:
  "registry.internal:5000":
    endpoint: ["http://registry.internal:5000"]
```

then `sudo systemctl restart k3s`: k3s regenerates the containerd configuration at start-up. On a managed cluster you do not touch the nodes: the registry **must** use TLS (a recognised certificate), and credentials go into a `kubernetes.io/dockerconfigjson` Secret referenced by `imagePullSecrets` in the Pod (or the ServiceAccount).

Why. The pull is done by the node's kubelet/containerd, with the node's configuration; the API server plays no part.

Nuance. The same file serves for mirrors (`"docker.io": endpoint: [...]`) and authentication (`configs: … auth: username/password`). An HTTP registry, even internal, is a lab exception: in a company, TLS everywhere.

Example.

```
kubectl create secret docker-registry reg --docker-server=registry.internal --docker-username=ci --docker-password=…
kubectl patch serviceaccount default -p '{"imagePullSecrets":[{"name":"reg"}]}'
```

Question 10 — `helm upgrade --install --wait`

Answer. `--install` makes the command **idempotent**: first run = `install`, later runs = `upgrade`, the same line in every case. `--wait` waits until every Pod is `Ready` (lab 12 probes), Services have endpoints, PVCs are bound; `--timeout 5m` bounds that wait, after which the release becomes `failed` and the job fails: CI **knows** the deployment did not converge. The runner should hold a kubeconfig tied to a **ServiceAccount** with an `edit` RoleBinding on the single target namespace (lab 16): never the k3s `cluster-admin` kubeconfig. On failure: `helm rollback api` (or `--atomic`, which does it by itself), then alert.

Why. Without `--wait`, `helm upgrade` returns as soon as the API server has accepted the objects, long before the Pods run.

Nuance. `--wait` only detects what the probes reveal: an honest readiness (lab 12) is the condition of its value. And a Secret holding the runner's kubeconfig is a secret like any other: rotation, minimal scope.

Example.

```
helm upgrade --install stack ./stack -n lab17 --create-namespace --wait --timeout 2m --atomic
helm history stack      # failed → automatic rollback with --atomic
```

Question 11 — GitOps

Answer. With "push" CI, a job runs `kubectl`/`helm` towards the cluster: it must be given credentials, and the cluster's real state ends up drifting from what Git holds (an emergency `kubectl edit`, a manual `helm rollback`). With "pull" GitOps, an agent **inside** the cluster (Argo CD, Flux) reads a Git repository and continuously applies what it describes; CI only builds, pushes the image and **changes the tag in Git**. Advantages: every change is a commit (audit, review through a *merge request*), rollback is a `git revert`, **drift** is detected and corrected (or reported). Consequence: CI has no access to the cluster at all. It is the lab 11 loop — desired state versus actual state — applied to the whole Git repository.

Why. The declarative model makes an external source of truth possible; the Kubernetes controller reconciles objects, the GitOps agent reconciles repositories.

Nuance. Secrets do not go into Git in clear text: Sealed Secrets, SOPS or External Secrets (lab 14). And an "emergency" change made by hand will be overwritten by the agent: that is the point.

Example.

```
git commit -am "api: 1.1 -> 1.2 in prod" && git push      # that is the whole deployment
kubectl get applications -n argocd                        # Synced / OutOfSync (Argo CD)
```

Question 12 — Towards a company cluster

Answer. On EKS/AKS/GKE: `Service: LoadBalancer` gets a cloud load balancer with a public IP (and a cost) instead of ServiceLB; `local-path` does not exist — the default StorageClass is a network disk (`gp3`, `managed-csi`, `pd-balanced`), which lets the Pod change nodes; the Ingress controller must be installed or replaced (nginx, ALB, GKE Ingress); identity goes through IAM/Entra/Google and RBAC set by the platform. On OpenShift, add **Routes** (instead of or beside Ingress), the **SCC** that refuse a root container by default (your lab 05 `USER` becomes mandatory, and the UID is imposed) and `oc`. Unchanged: Pods, Deployments, Services, ConfigMaps/Secrets,

PVCs, probes, RBAC, the chart itself. To stay portable: do not hard-code a StorageClass (leave the default, or a value), make the Ingress and its annotations parameterisable, never assume a node port, run non-root, and declare `resources`.

Why. The API is standard; the integrations (network, storage, identity) are provided by each platform behind the same objects.

Nuance. `hostPath`, `hostPort`, `NodePort` and anything assuming "a single node" break first. Test the chart with `helm template -f values-cloud.yaml` before you have the cluster.

Example.

```
# values.yaml — portable
db: { storageClass: "" }        # "" = the cluster's default StorageClass
web: { ingress: { enabled: true, className: "", annotations: {} } }
```

Question 13 — `latest` + `rollout restart`

Answer. Reproducibility: `latest` names a different image at every push; two Pods of the same Deployment may run two versions, and nobody knows which. Rollback: `rollout undo` brings back an identical template (`latest`), hence **the same broken image**; without a tag there is nothing to go back to. Audit: no commit, no Helm revision says what is running; `imageID` is the only witness. Security: `Always` + `latest` means whoever can push to that tag deploys to production without going through CI. Correct version: CI builds `api:<git-sha>` (immutable), pushes, then `helm upgrade --install … --set api.image.tag=<sha> --wait --atomic` (or changes the tag in Git for Argo CD); in production, the digest; `imagePullPolicy: IfNotPresent` is then enough, since the tag never moves.

Why. A deployment must be a function of its inputs (chart + values + digest); `latest` removes the most important input.

Nuance. `latest` remains acceptable… for a local try on k3s, never beyond. And `rollout restart` is useful — to reload a config, not to deliver a version.

Example.

```
helm upgrade --install stack ./stack --set api.image.tag=$(git rev-parse --short HEAD) --wait --atomic
helm history stack      # every revision names a precise image
```